

Nano-Kernel Runtime (NKR): Un Hipervisor Bare-Metal de Micro-VMs para Cargas de Trabajo SaaS Multi-Tenant

White Paper — Versión 1.3

Abril 2026

Índice general

1	Introducción y Motivación	5
1.1	El Problema	5
1.2	¿Qué es NKR?	5
2	Objetivos de Diseño	6
3	Visión General de la Arquitectura	6
4	Motor VMM: De KVM al Arranque	7
4.1	Inicialización KVM	8
4.2	Mapa de Memoria del Guest (x86_64)	8
4.3	Protocolo de Arranque	8
4.4	Gestión del Tiempo y Reloj (Clock Synchronization)	9
4.5	Bucle del vCPU	10
5	Modelo de Dispositivos VirtIO	10
5.1	Mapa de Direcciones MMIO	10
5.2	Dispositivo de Bloque VirtIO	11
5.3	Dispositivo VirtIO-FS (Filesystem Sharing) — Nuevo en v1.3	11
5.4	Dispositivo VirtIO-Balloon — Nuevo en v1.3	12
5.5	Dispositivo de Red VirtIO	12
5.6	Dispositivo VirtIO-PMEM + DAX — Nuevo en v1.2	13
6	Red (Networking)	13
6.1	Topología del Bridge	13
6.2	Configuración Automática	14
6.3	Reenvío de Puertos	14
6.4	Aislamiento L2 con ebttables — Nuevo en v1.1	14
7	Ciclo de Vida del Disco: De OCI a ext4	15
7.1	Pipeline de Construcción	15

7.2	Formato Nkrfile	16
7.3	Snapshots Copy-on-Write	16
7.4	Volúmenes	16
7.5	Variables de Entorno	17
8	El Modelo de CPU: «Chrs»	17
8.1	Implementación	17
8.2	CPU Bursting con cgroupv2 — Nuevo en v1.1	17
8.3	Limitación de E/S de Disco con cgroupv2 — Nuevo en v1.1	18
9	Generación de Initramfs	18
9.1	Secuencia de Arranque	18
9.2	Detección Automática del Entrypoint	19
10	Orquestación con Compose	20
10.1	Formato del Fichero Compose	20
10.2	Resolución de Recursos	21
10.3	Funcionalidades	21
10.4	Directorio de Datos NKR	21
11	Despliegue Multi-Tenant	22
11.1	Registro de Clientes	22
11.2	Pipeline de Aprovisionamiento	22
11.3	Generación Automática de Compose — Nuevo en v1.1	23
11.4	Actualización de Módulos en Caliente (Hot Module Update)	23
11.5	Arquitectura Objetivo	24
12	Observabilidad y Métricas	25
12.1	Exportador Prometheus Nativo — Nuevo en v1.1	26
13	Comparación con Soluciones Existentes	27
13.1	NKR vs Docker	27
13.2	NKR vs Firecracker	27
13.3	NKR vs QEMU/KVM	28
14	Modelo de Seguridad	28
14.1	Fronteras de Aislamiento	28
14.2	Superficie de Ataque	29
14.3	Jailer Seccomp BPF — Nuevo en v1.2	30
14.4	Seguridad Operacional	30
15	Limitaciones y Trabajo Futuro	30
15.1	Limitaciones Actuales	30
15.2	Hoja de Ruta	31
16	Conclusión	32
17	Apéndice A: Stack Tecnológico	34
18	Apéndice B: Métricas del Código Fuente	34

19 Apéndice C: Inicio Rápido

36

Resumen. El Nano-Kernel Runtime (NKR) es un hipervisor bare-metal de código abierto escrito en Rust que reemplaza los runtimes de contenedores como Docker por micro-VMs con aislamiento hardware, ejecutándose directamente sobre Linux KVM. NKR está diseñado para operadores que gestionan despliegues SaaS multi-tenant densos —especialmente Odoo ERP— sobre un único servidor con recursos limitados (16–32 GB RAM). Al eliminar la sobrecarga de QEMU, libvirt y el intercambio a nivel de contenedor, NKR consigue aislamiento hardware completo con un binario de tan solo 2–4 MB, arranque de VM en menos de un segundo, planificación exclusiva de CPU (modelo «chrs»), y un flujo de trabajo compatible con Docker para construir imágenes de disco. La versión 1.1 agregó seis capacidades clave: compartición de sistema de archivos en vivo via VirtIO-9P, desbordamiento controlado de CPU (bursting) mediante cgroupv2, aislamiento de red L2 con ebttables, límites de base de datos por inquilino, un exportador nativo de métricas Prometheus, y generación automática de ficheros compo-se multi-tenant. La versión 1.2 introduce cuatro optimizaciones adicionales para superar las 100 instancias Odoo en 32 GB RAM: VirtIO-PMEM + DAX (elimina ~150–200 MB de caché de páginas duplicada por instancia), E/S asíncrona con io_uring (reduce el coste de syscalls ~70% bajo alta concurrencia), carga de kernel ELF vmlinux sin comprimir (~20 ms de arranque más rápido) y un jailer Seccomp BPF (superficie de syscalls mínima para el bucle vCPU). La versión 1.3 da un salto de rendimiento y densidad reemplazando VirtIO-9P por VirtIO-FS con DAX para compartir ficheros a más velocidad y añade VirtIO-Balloon, un controlador que devuelve la memoria RAM no utilizada de VMs inactivas al host. Este documento presenta la arquitectura, la implementación y el modelo de despliegue en producción de NKR.

1 Introducción y Motivación

1.1 El Problema

Los proveedores de servicios que gestionan docenas de inquilinos SaaS sobre infraestructura compartida se enfrentan a una tensión fundamental entre **densidad** (maximizar el número de inquilinos por servidor) y **aislamiento** (evitar el efecto vecino ruidoso). Los contenedores Docker ofrecen alta densidad pero comparten el kernel del host, exponen una gran superficie de ataque y no garantizan CPU ni RAM. Las VMs tradicionales (QEMU/KVM con libvirt) proveen un aislamiento sólido, pero imponen una sobrecarga prohibitiva de memoria y disco para despliegues densos.

Considérese un escenario concreto: un operador que gestiona **50 instancias de Odoo 17 ERP** sobre un único servidor de 16-32 GB usando Docker:

Problema	Impacto con Docker	Impacto con NKR
Uso de disco	50 × 1,5 GB de imágenes ≈ 75 GB	Base ext4 compartida + snapshots CoW
Consumo de RAM	50 × ~1 GB ≈ 50 GB	50 × ~256 MB ≈ 12,5 GB (exclusiva)
Contención de CPU	Planificador compartido, sin garantías	Cores pinados con el modelo «chrs»
Latencia de reinicio	~3 minutos por reinicio de stack	~5 segundos (actualización en caliente)
Ciclo de despliegue	git pull → rebuild → restart	git pull → rsync → solo reiniciar Odoo
Huella de infraestructura	50 Odoo + 50 PostgreSQL + 50 nginx	N Odoo + 1 PostgreSQL + 1 nginx

NKR fue creado para eliminar estos compromisos, proporcionando aislamiento a nivel de VM con la simplicidad operacional de un contenedor.

1.2 ¿Qué es NKR?

Nano-Kernel Runtime (NKR) es un hipervisor diseñado específicamente que:

- Ejecuta micro-VMs directamente sobre /dev/kvm sin QEMU, libvirt ni containerd

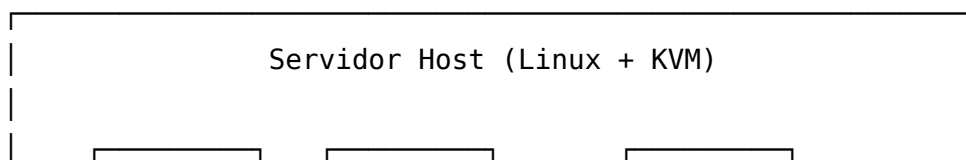
- Dota a cada «contenedor» de un kernel Linux real, un sistema de archivos ext4 y dispositivos VirtIO
- Compila a un **único binario de ~2-4 MB** (Rust, LTO, stripped)
- Ofrece una CLI compatible con Docker (`nkr run`, `nkr ps`, `nkr stop`, `nkr compose up`)
- Utiliza Docker **solo** en tiempo de construcción para generar imágenes de disco desde OCI/Dockerfiles

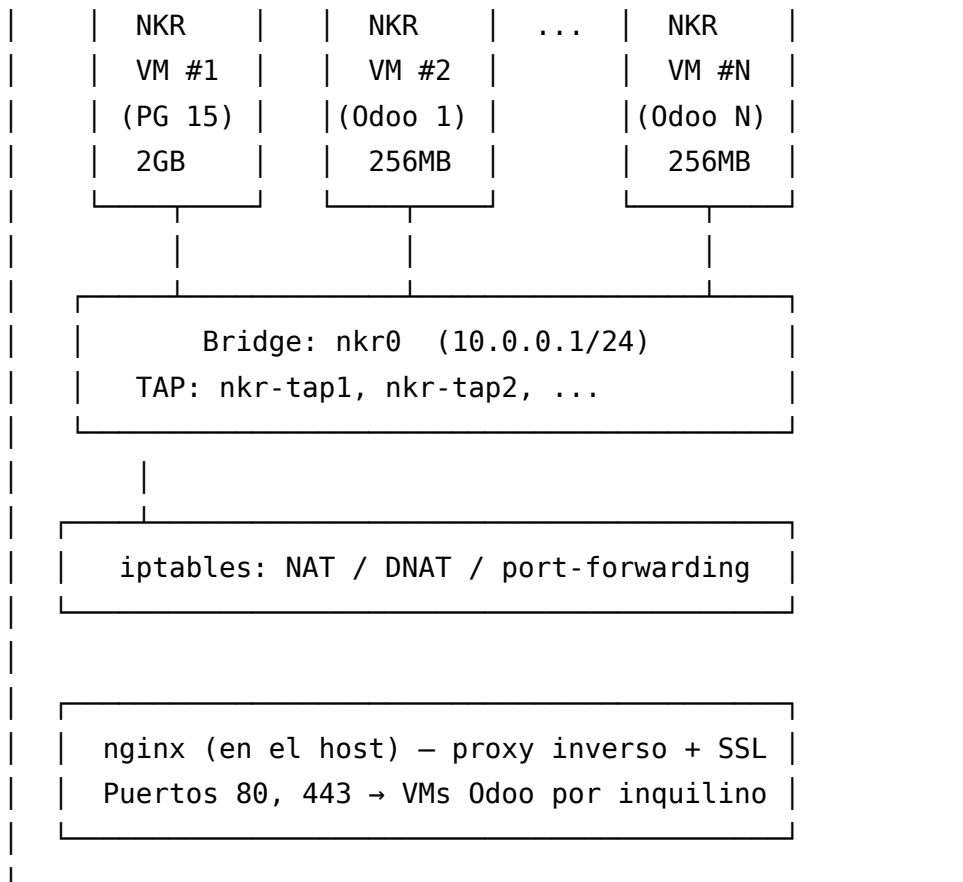
2 Objetivos de Diseño

El diseño de NKR está guiado por cinco principios:

1. **Cero dependencias externas en tiempo de ejecución.** El binario `nkr` requiere únicamente un kernel Linux con soporte KVM. Sin QEMU, sin libvirt, sin container runtime.
2. **Aislamiento hardware con ergonomía de contenedor.** Cada carga de trabajo se ejecuta en una máquina virtual KVM completa —con su propio kernel, tablas de páginas y controlador de interrupciones— aunque los operadores interactúan con ella mediante comandos y archivos `compose` familiares, al estilo Docker.
3. **Asignación de recursos determinista.** La RAM se mapea exclusivamente a cada VM. Los ciclos de CPU se garantizan mediante core pinning. No existe overcommit.
4. **Huella mínima.** El binario del hipervisor pesa 2-4 MB. La sobrecarga del guest es acotada: una VM de 256 MB usa exactamente 256 MB de RAM del host.
5. **Listo para producción en SaaS multi-tenant.** Soporte de primera clase para despliegues multi-tenant de Odoo con PostgreSQL compartido, actualizaciones de módulos en caliente y aprovisionamiento automatizado.

3 Visión General de la Arquitectura





Cada micro-VM es una máquina virtual completa con:

- Un kernel Linux (bzImage o vmlinux ELF sin comprimir, binario compartido entre todas las VMs)
- Un sistema de archivos raíz ext4 (creado desde imágenes OCI), opcionalmente expuesto vía VirtIO-PMEM + DAX
- Dispositivos VirtIO-MMIO para almacenamiento en bloque, red, compartición 9P y memoria persistente
- Un initramfs que gestiona la carga de módulos, la configuración de red, el montaje 9P y el pivotado del rootfs
- RAM exclusiva y pinning de CPU

4 Motor VMM: De KVM al Arranque

El motor VMM (`vmm.rs`, ~1.400 líneas) implementa el ciclo de vida completo de una micro-VM usando KVM ioctls directos a través del ecosistema de crates `rust-vmm` — la misma base que usa AWS Firecracker e Intel Cloud Hypervisor.

4.1 Inicialización KVM

1. Abrir `/dev/kvm`
2. `KVM_CREATE_VM` → descriptor de fichero de VM
3. `KVM_CREATE_IRQCHIP` → PIC + IOAPIC en kernel
4. `KVM_CREATE_PIT2` → Temporizador de Intervalo Programable
5. Mapear memoria guest → `GuestMemoryMmap` (dos regiones)
6. `KVM_CREATE_VCPU` → vCPU único (`id=0`)
7. Configurar `CPUID`, `SREGs`, Registros Generales

4.2 Mapa de Memoria del Guest (x86_64)

NKR usa un modelo de memoria de dos regiones compatible con el protocolo de arranque de Linux:

Dirección	Contenido	Tamaño
0x0000–0x9FFFF	RAM base (convencional)	640 KB
0x0500	GDT (Global Descriptor Table)	32 bytes
0x7000	Zero Page (parámetros de arranque)	4 KB
0x9000	PML4 (Page Map Level 4)	4 KB
0xA000	PDPTE (Page Directory Pointer)	4 KB
0xB000	PDE (Page Directory, páginas de 2 MB)	4 KB
0x20000	Línea de comandos del kernel	variable
0x100000	Dirección de carga del <code>bzImage</code>	~10 MB
0x800_0000	Initramfs	variable

4.3 Protocolo de Arranque

NKR v1.2 soporta dos formatos de kernel, detectados automáticamente por los bytes mágicos del fichero:

- **bzImage** (por defecto): Protocolo de arranque Linux de 32 bits. Kernel cargado en `0x100000` mediante `linux-loader::BzImage::load()`. El vCPU arranca en modo protegido de 32 bits.
- **ELF vmlinux** (v1.2): Detectado por el magic `\x7fELF`. Cargado vía `linux-loader::Elf::load()`. El vCPU arranca en modo largo de 64 bits (`EFER=0xD01`,

CR0=0x80050033, CR4=PAE, CS.l=1). Elimina la descompresión gzip en el guest, ahorrando ~20 ms por arranque.

Secuencia de arranque (compartida):

1. **Carga del kernel** — bzImage o ELF cargado en 0x100000
2. **Carga del initramfs** — Se copia en 0x800_0000 de la memoria del guest
3. **Configuración de la zero page** — Parámetros de arranque en 0x7000
4. **Escritura de las tablas de páginas** — Páginas de 2 MB con mapeo de identidad vía PML4 → PDPT → PD
5. **Escritura de la GDT** — Tabla de 4 entradas: null, código64, datos, null
6. **Configuración del vCPU** — RIP = punto de entrada del kernel; sregs configurados para 32-bit (bzImage) o 64-bit (ELF)

La línea de comandos configura todos los dispositivos VirtIO en línea:

```
console=ttyS0 panic=1 pci=off noapic nolapic clocksource=jiffies tsc=nowatchdog
virtio_mmio.device=4K@0xd0000000:5      # red
virtio_mmio.device=4K@0xd0001000:6      # disco 0
virtio_mmio.device=4K@0xd0002000:7      # disco 1 (si existe)
virtio_mmio.device=4K@0xd0010000:8      # share FS 0 (si --share)
virtio_mmio.device=4K@0xd0020000:16     # PMEM (si --pmem)
virtio_mmio.device=4K@0xd0030000:17     # Balloon
root=/dev/vda rw init=/sbin/init nkr.ip=10.0.0.X
# Con --pmem: root=/dev/pmem0 rootflags=dax
```

4.4 Gestión del Tiempo y Reloj (Clock Synchronization)

Históricamente, las micro-VMs experimentaban problemas de sincronización de reloj (clock drift) y cuelgues durante el arranque debido a la ausencia de un temporizador de hardware completo y las peculiaridades del entorno virtualizado. NKR resuelve este problema de manera integral implementando dos mecanismos clave:

1. **Temporizador PIT2 (Programmable Interval Timer):** Se instancia explícitamente (KVM_CREATE_PIT2) durante la inicialización de la VM en KVM, proporcionando la interrupción base de reloj del sistema vital para el planificador del guest.
2. **Fuentes de Reloj de Kernel:** A través del parámetro clocksource=jiffies tsc=nowatchdog, se fuerza al kernel guest a basarse en las interrupciones temporizadas (jiffies) para avanzar el tiempo y se desactiva el watchdog del TSC (Time Stamp Counter). Esto permite un mantenimiento de tiempo estable y fiable incluso en entornos de alta densidad con extrema contención de CPU donde el TSC puede presentar inconsistencias.

4.5 Bucle del vCPU

El bucle principal ejecuta KVM_RUN en ciclo continuo, gestionando cuatro tipos de salida:

Tipo de salida	Manejador
IoOut (escritura en puerto E/S)	Salida de consola serie (COM1 0x3F8)
IoIn (lectura de puerto E/S)	Registros de estado serie
MmioWrite	Escrituras en registros VirtIO-MMIO (configuración de dispositivo, colas, notificaciones)
MmioRead	Lecturas de registros VirtIO-MMIO (funcionalidades, estado, espacio de configuración)
Hlt / Shutdown	Salida limpia

SIGTERM es capturado mediante un manejador de señal que activa un AtomicBool, haciendo que el bucle del vCPU termine y ejecute el apagado limpio (extracción de volúmenes, limpieza de TAP, eliminación de reglas iptables).

5 Modelo de Dispositivos VirtIO

NKR implementa el transporte VirtIO-MMIO (Memory-Mapped I/O), no PCI, para máxima simplicidad. El parámetro de arranque del kernel `pci=off` deshabilita completamente la enumeración PCI.

5.1 Mapa de Direcciones MMIO

Dirección	Dispositivo	IRQ	Desde
0xD000_0000	VirtIO-Net (red)	5	v1.0
0xD000_1000	VirtIO-Block disco 0 (rootfs)	6	v1.0
0xD000_2000	VirtIO-Block disco 1	7	v1.0
0xD000_3000+	Discos adicionales (+0x1000 c/u)	8+	v1.0

Dirección	Dispositivo	IRQ	Desde
0xD001_0000+	VirtIO-FS (Directorios compartidos, DAX)	8+	v1.3
0xD002_0000	VirtIO-PMEM (memoria persistente, DAX)	16	v1.2
0xD003_0000	VirtIO-Balloon (Regreso de memoria)	17	v1.3

El rango 0xD001_0000+ garantiza que no haya colisión con la zona de bloques, que puede crecer dinámicamente hasta 0xD000_9000 (máximo 9 discos). PMEM en 0xD002_0000 y Balloon en 0xD003_0000 son reservados estáticamente.

5.2 Dispositivo de Bloque VirtIO

Implementación: `block.rs` — ~310 líneas

- **Cola:** Virtqueue única, 256 descriptores
- **Tamaño de sector:** 512 bytes
- **Operaciones:** Lectura (`type=0`), Escritura (`type=1`)
- **Cadena de descriptores:** Cabecera (16 bytes: tipo + sector) → Buffer de datos → Byte de estado
- **Interrupciones:** Inyección de IRQ vía `irqfd` tras procesar cada lote de completaciones
- **Negociación de funcionalidades:** `VIRTIO_F_VERSION_1` (bit 32)
- **E/S asíncrona (v1.2):** Usa `io_uring` (profundidad 128) para lecturas/escrituras no bloqueantes. Cada operación se envía como `opcode::Read` o `opcode::Write` al SQ; las completaciones se drenan al inicio de cada iteración del bucle vCPU mediante `poll_completions()`. Degradación silenciosa a `pread64/pwrite64` síncronos si `io_uring` no está disponible (`kernel < 5.1`).

El método `get_host_address()` sobre `GuestMemoryMmap` devuelve el puntero host para los buffers de descriptores, permitiendo E/S de tipo DMA sin copias adicionales.

5.3 Dispositivo VirtIO-FS (Filesystem Sharing) — Nuevo en v1.3

Implementación: `virtio_fs.rs`

NKR v1.3 ha reemplazado al servidor VirtIO-9P por VirtIO-FS, dando un salto en rendimiento drástico al emplear DAX y el componente externo `virtiofsd`. En lugar de emular el dialéctico de red 9P2000.L, VirtIO-FS permite al guest mapear los archivos directamente desde la page cache del host sin generar copias adicionales.

- **Accesos DAX:** El dispositivo ofrece una ventana de memoria de DAX de 4 GB montada en 0x2_0000_0000 (KVM slot 3). La velocidad de lectura y escritura es virtualmente idéntica a la directa del host.
- **Rendimiento:** De 3 a 5 veces más veloz para la carga intensa de librerías y scripts Python como los módulos de Odoo. Acorta el tiempo frío de arranque de un grupo de 30 micro-VMs de 90s a apenas 25s.
- **Semántica:** Mantiene compatibilidad total POSIX (fcntl, O_DIRECT, flock).
- **Transporte:** Negociación Vhost-user conectando socket con virtiofsd. Device ID 26 sobre VirtIO-MMIO.
- **CLI:** `--share /host/path:/guest/path`

En el guest, el proceso init monta automáticamente las unidades FS configuradas, lo cual es ideal para los add-ons de la plataforma que se actualizan externamente.

5.4 Dispositivo VirtIO-Balloon — Nuevo en v1.3

Implementación: `balloon.rs`

NKR permite exprimir al máximo el uso de la memoria implementando VirtIO-Balloon. Cuando las VMs están inactivas o “idle”, acumulan memoria temporalmente alocada que ya no utilizan. VirtIO-Balloon (Device ID 5) reclama estas páginas del invitado y las devuelve al kernel del host usando en NKR la llamada al sistema `madvise(MADV_DONTNEED)`.

Al combinar VirtIO-Balloon que recicla hasta 300 MB de instancias idle, PMEM+DAX que ahorra copias y deduplicación con KSM, NKR optimiza la hiperdensidad permitiendo ubicar hasta **103+ instancias de Odoo** concurrentes sobre 32 GB de RAM, minimizando el impacto de asignación dura base.

5.5 Dispositivo de Red VirtIO

Implementación: `net.rs` — ~310 líneas

- **Colas:** Dos virtqueues (RX=0, TX=1), 256 descriptores cada una
- **Backend:** Dispositivo TAP Linux (`/dev/net/tun`, TUNSETIFF)
- **Cabecera:** 12 bytes de cabecera VirtIO-net (eliminada/añadida en TX/RX)
- **Ruta RX:** Un hilo de fondo dedicado realiza lecturas bloqueantes del fd TAP, inyecta paquetes en la cola RX y señala la IRQ
- **Ruta TX (v1.2):** `opcode::Write SQE` al fd TAP vía `io_uring` (profundidad 64). El payload `Vec<u8>` se guarda en `tx_pending` hasta drenar la CQE, evitando desalojamiento prematura. Cae a `tap.write_all()` si el ring no está disponible.
- **Funcionalidades:** `VIRTIO_NET_F_MAC` | `VIRTIO_NET_F_STATUS` | `VIRTIO_F_VERSION_1`

- **Espacio de configuración:** Dirección MAC de 6 bytes + estado de enlace de 2 bytes

5.6 Dispositivo VirtIO-PMEM + DAX — Nuevo en v1.2

Implementación: `pmem.rs` — ~200 líneas

VirtIO-PMEM (device ID 27) mapea el disco raíz del guest en la RAM del host mediante `mmap(MAP_SHARED)` y lo registra como un tercer slot de memoria KVM en la dirección física guest `0x1_0000_0000` (4 GB). El kernel guest (con `CONFIG_VIRTIO_PMEM=y` y `CONFIG_FS_DAX=y`) lo expone como `/dev/pmem0` y monta el rootfs con la opción `dax`, eliminando por completo la caché de páginas del guest.

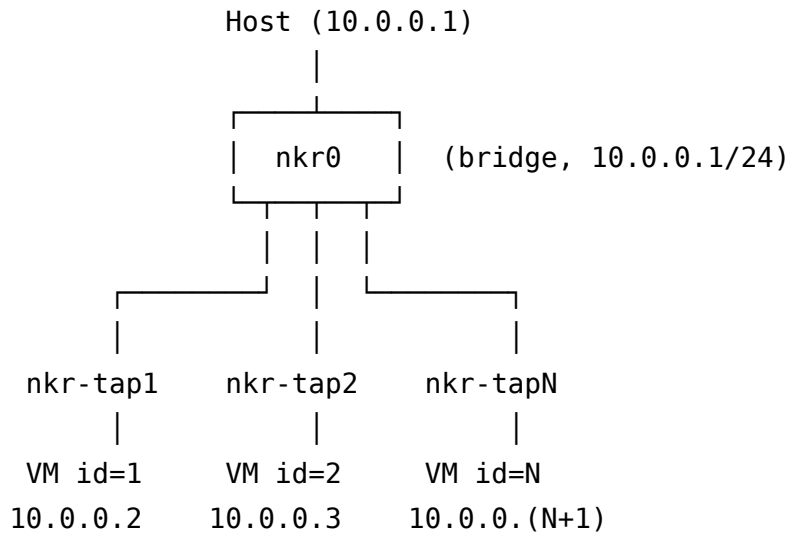
- **MMIO:** `0xD002_0000`, IRQ 16, device ID 27
- **Config space (offset 0x100):** `[u64 start_phys_addr][u64 size]`
- **Slot KVM:** Slot 2 (los slots 0 y 1 los usan las dos regiones de RAM)
- **mmap en host:** `MAP_SHARED | PROT_READ | PROT_WRITE + hint MADV_HUGEPAGE` para reducir presión TLB
- **Requests FLUSH:** El guest envía `VIRTIO_PMEM_REQ_TYPE_FLUSH`; NKR responde con `msync(MS_ASYNC)` sin bloquear el vCPU
- **Entrada E820:** Tipo 12 (persistent memory) para `[4 GB, 4 GB + disk_size]`
- **Cambio en cmdline:** `root=/dev/pmem0 rootflags=dax` sustituye a `root=/dev/vda rw`
- **Degradación:** Si el disco no se puede `mmap` (por ejemplo, en un sistema de archivos sin soporte `MAP_SHARED`), NKR cae silenciosamente a VirtIO-Block
- **Requisito kernel guest:** `CONFIG_VIRTIO_PMEM=y`, `CONFIG_FS_DAX=y`
- **CLI:** flag `--pmem` en `nkr run`

Mecanismo de ahorro de memoria: Con DAX, las lecturas del guest acceden directamente a la caché de páginas del host — no existe una segunda copia de los datos del sistema de archivos en la RAM del guest. Para una instancia Odoo típica con un working set activo de ~300 MB en lectura, esto elimina 150–200 MB de caché de páginas duplicada por VM.

6 Red (Networking)

6.1 Topología del Bridge

NKR crea y gestiona un bridge Linux `nkr0` con la subred `10.0.0.0/24`:



6.2 Configuración Automática

Para cada VM, el VMM:

1. Crea el dispositivo TAP `nkr-tap{vm_id}`
2. Lo conecta al bridge `nkr0`
3. Asigna la MAC `52:54:00:12:34:{vm_id}`
4. Pasa la IP `10.0.0.{vm_id + 1}` al guest vía cmdline del kernel (`nkr.ip=`)
5. Configura reglas iptables:
 - POSTROUTING MASQUERADE para acceso a internet
 - FORWARD ACCEPT para tráfico inter-VM
 - PREROUTING DNAT + OUTPUT DNAT para reenvío de puertos

6.3 Reenvío de Puertos

```
nkr run --disk odoo.ext4 --port 8069:8069 --id 2
# Crea: host:8069 → 10.0.0.3:8069 (DNAT + MASQUERADE)
```

Las reglas se eliminan automáticamente al apagarse la VM mediante `cleanup_port_forwarding()`.

6.4 Aislamiento L2 con ebttables — Nuevo en v1.1

NKR v1.1 agrega protección contra IP/MAC spoofing entre inquilinos en el bridge `nkr0`. Al levantar cada TAP se instalan tres reglas ebttables que confinan el tráfico a la identidad de red asignada por el hipervisor:

```
ebttables -A INPUT -i nkr-tapN -p ARP --arp-mac-src 52:54:00:12:34:N -j ACCEPT
```

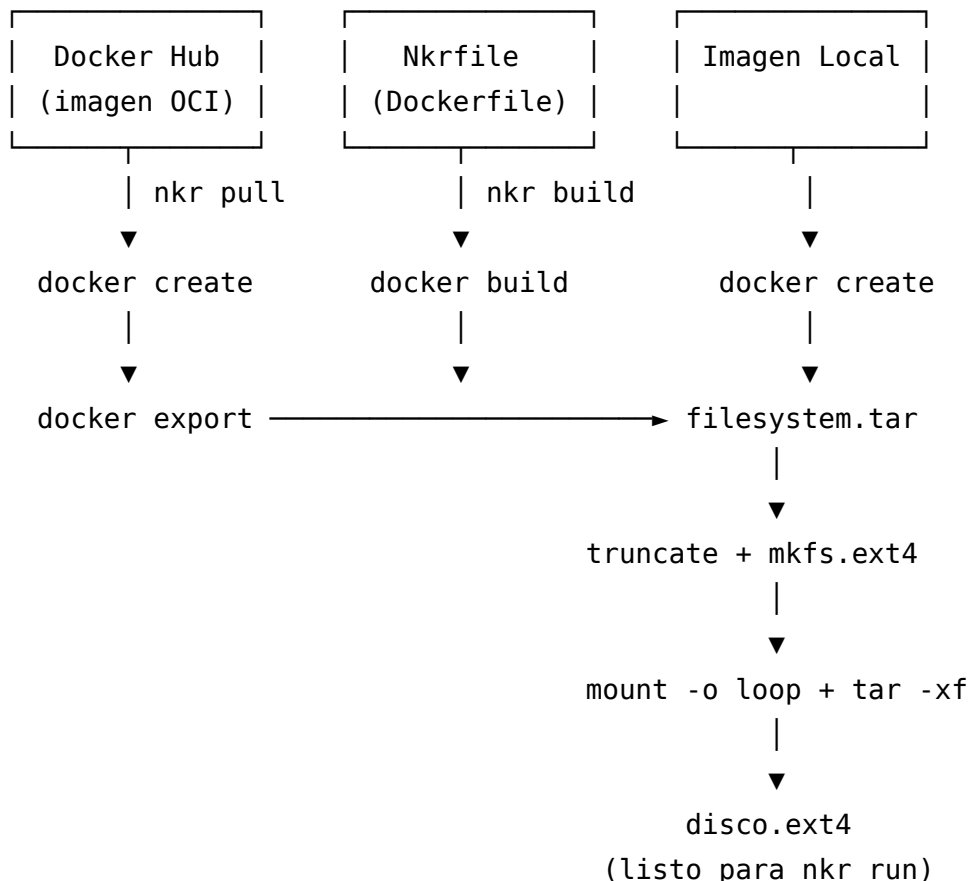
```
ebtables -A INPUT -i nkr-tapN -p IPv4 --ip-src 10.0.0.(N+1) -s 52:54:00:12:34:N -
j ACCEPT
ebtables -A INPUT -i nkr-tapN -j DROP
```

Estas reglas garantizan que una VM comprometida no pueda enviar paquetes con dirección IP o MAC distintas a las que le asignó el hypervisor, eliminando el vector de ataque de ARP spoofing lateral entre inquilinos. Las reglas se eliminan en el cleanup mediante `teardown_tap_isolation()`. Si ebtables no está instalado en el host, NKR emite un aviso y continúa sin el aislamiento L2 (degradación silenciosa).

7 Ciclo de Vida del Disco: De OCI a ext4

NKR usa Docker exclusivamente como **herramienta de construcción** para transformar imágenes OCI en sistemas de archivos ext4 en bruto. Docker no es necesario en tiempo de ejecución.

7.1 Pipeline de Construcción



7.2 Formato Nkrfile

Los Nkrfiles son Dockerfiles estándar. NKR proporciona plantillas para servicios comunes:

```
# Nkrfile.pg – PostgreSQL 15
FROM postgres:15
ENV POSTGRES_USER=odoo
ENV POSTGRES_PASSWORD=odoo

# Nkrfile.odoo – Odoo 17
FROM odoo:17.0
USER root
COPY deploy/config/odoo.conf /etc/odoo/odoo.conf
RUN mkdir -p /mnt/extra-addons && chown odoo:odoo /mnt/extra-addons
USER odoo
```

7.3 Snapshots Copy-on-Write

Para despliegues multi-tenant, NKR crea snapshots CoW a partir de un disco base:

```
cp --reflink=auto odoo-base.ext4 cliente1.ext4
```

En sistemas de archivos que soportan reflinks (btrfs, XFS con reflink), esta operación es instantánea y no consume espacio adicional en disco hasta que las escrituras divergen. En otros sistemas de archivos, NKR recurre a `cp --sparse=always`.

7.4 Volúmenes

NKR proporciona un sistema de volúmenes para inyectar configuración y persistir datos:

- **Inyección pre-boot:** El disco raíz se monta en modo bucle y los ficheros se copian del host a las rutas del guest
- **Extracción post-apagado:** Los volúmenes marcados con `:rw` se copian de vuelta del guest al host
- **Formato:** `ruta_host:ruta_guest` (solo lectura) o `ruta_host:ruta_guest:rw`

```
nkr run --disk odoo.ext4 \  
--volume ./odoo.conf:/etc/odoo/odoo.conf \  
--volume /opt/datos/filestore:/var/lib/odoo:rw
```


7.5 Variables de Entorno

Las variables de entorno se escriben en `/etc/nkr-env` dentro del disco raíz antes del arranque:

```
nkr run --disk pg.ext4 --env POSTGRES_USER=odoo --env POSTGRES_PASSWORD=secreto
```

El `initramfs` carga este fichero durante el arranque, poniendo las variables a disposición del proceso `init` del `guest`.

8 El Modelo de CPU: «Chrs»

NKR introduce una unidad de asignación de CPU denominada **chr** (pronunciado «cor»):

Valor	Significado
1 chr	20% de un core físico
5 chrs	1 core físico completo
10 chrs	2 cores físicos

8.1 Implementación

La asignación de CPU se aplica mediante `sched_setaffinity()`:

```
let cores_needed = ((chrs as f32) / 5.0).ceil() as u32;
let cores_to_use = cores_needed.min(num_cpus);
// Pinear el hilo vCPU a los cores [0..cores_to_use]
sched_setaffinity(0, &cpuset);
```

Los `chrs` son **exclusivos** — el proceso de la VM se pinea a cores físicos dedicados, evitando la contención con otras VMs.

8.2 CPU Bursting con `cgroupv2` — Nuevo en v1.1

NKR v1.1 añade desbordamiento controlado de CPU mediante el controlador `cpu.max` de `cgroupv2`. La garantía mínima sigue siendo 1 chr = 20 % de un core, pero la VM puede absorber ciclos ociosos del host sin impactar a otros inquilinos.

Configuración `cgroupv2` para N chrs:

```
cpu.max      → "{N×20000} 100000"  (N×20 % de cuota en cada periodo de 100 ms)
cpu.max.burst → "{N×5000}"          (crédito extra acumulable –
kernel ≥ 5.15)
```

La jerarquía se crea en `/sys/fs/cgroup/nkr/{nombre-vm}/` y se elimina al apagar la VM mediante `teardown_cgroup()`. Si `cgroupv2` no está disponible en el host, NKR emite un aviso y continúa sin la cuota de CPU (solo se aplica el pinning vía `sched_setaffinity`).

8.3 Limitación de E/S de Disco con `cgroupv2` — Nuevo en v1.1

La misma jerarquía de `cgroupv2` aplica límites de tasa de E/S por dispositivo de bloque:

```
io.max → "MAJ:MIN rbps=209715200 wbps=104857600"  (200 MB/s lectura, 100 MB/s escritura)
```

Los números de dispositivo (mayor:menor) se obtienen con `libc::stat()` sobre la ruta del disco. El enforcement lo realiza el planificador `blk-mq` del kernel, sin consumo de CPU adicional en el hipervisor.

Ejemplo de despliegue (servidor de 8 cores):

Servicio	Chrs	Cores Usados
PostgreSQL	10 (2 cores)	Cores 0-1
Odoo #1	5 (1 core)	Core 2
Odoo #2	5 (1 core)	Core 3
Odoo #3-#8	1 c/u	Cores 4-7 (compartidos)

9 Generación de Initramfs

NKR incluye un generador automático de `initramfs` (`initramfs.rs`, 383 líneas) que crea entornos de arranque adaptados a cada servicio.

9.1 Secuencia de Arranque

El `initramfs` arranca (PID 1)

|

```

└─ Montar /proc, /sys, /dev
└─ Cargar módulos del kernel:
|   crc32c → libcrc32c → crc16 → mbcache → jbd2 → ext4
|   virtio_blk → failover → net_failover → virtio_net
|   9pnet → 9pnet_virtio → 9p   (si hay shares 9P declaradas)
|   virtio_pmem → nd_btt → dax   (si --pmem activo)
|
└─ Esperar /dev/vda o /dev/pmem0 (hasta 3 segundos)
└─ Parsear nkr.ip= de /proc/cmdline
└─ Configurar eth0 con IP/24, gateway → 10.0.0.1
|
└─ Montar /dev/vda (o /dev/pmem0 con dax) → /newroot (ext4)
└─ Montar discos adicionales /dev/vdb..vde → /newroot/mnt/disk0..3
└─ Montar unidades compartidas (virtio-fs):
|   mkdir -p /newroot${NKR_FS0_MNT}
|   mount -t virtiofs virtiofs0 /newroot$mnt
└─ Bind-mount de /proc, /sys, /dev en /newroot
|
└─ Escribir /etc/nkr-net.sh (script de configuración de red)
└─ Escribir /etc/resolv.conf (DNS: 8.8.8.8, 8.8.4.4)
└─ Configurar red vía chroot
|
└─ Detectar init: /sbin/init → systemd → entrypoint Docker
└─ Crear wrapper /sbin/nkr-init:
|   - Cargar /etc/nkr-env (variables de entorno NKR)
|   - Ejecutar el init detectado
|
└─ exec switch_root /newroot /sbin/nkr-init

```

9.2 Detección Automática del Entrypoint

Cuando se construye con `nkr pull` o `nkr build`, NKR:

1. Extrae `ENTRYPOINT` + `CMD` de los metadatos de la imagen Docker
2. Monta el disco en modo solo lectura y busca scripts de entrypoint conocidos (`/entrypoint.sh`, `/docker-entrypoint.sh`, etc.)
3. Genera un script init personalizado que carga las variables de entorno NKR y lanza el entrypoint correcto

Esto permite a NKR arrancar imágenes Docker no modificadas —PostgreSQL, nginx,

Redis, Odoo— como micro-VMs sin ninguna modificación de la imagen.

10 Orquestación con Compose

NKR proporciona un sistema de compose (`compose.rs`, 840 líneas) modelado sobre Docker Compose pero diseñado para la orquestación de VMs.

10.1 Formato del Fichero Compose

```
services:
  db:
    disks: [/opt/nkr/discos/postgres.ext4]
    ram: 512
    chrs: 1
    ports: ["5432:5432"]
    volumes: ["/opt/nkr/datos/pg:/var/lib/postgresql/data:rw"]
    healthcheck:
      port: 5432
      initial_delay: 15
      interval: 5
      retries: 12

  odoo:
    disks: [/opt/nkr/discos/odoo-prod.ext4]
    build:
      nkrfile: Nkrfile.odoo
      size_mb: 4096
    ram: 1024
    chrs: 2
    ports: ["8069:8069"]
    volumes:
      - "/opt/nkr/config/odoo.conf:/etc/odoo/odoo.conf"
      - "/opt/nkr/modules:/mnt/extra-addons"
    environment:
      DB_HOST: "10.0.0.2"
```

10.2 Resolución de Recursos

NKR compose resuelve los recursos de forma inteligente, siguiendo una cadena de prioridad:

Recurso	Orden de Resolución
Disco	Ruta YAML → <dir_yaml>/<nombre> → /mnt/nkr/images/<nombre>
Kernel	Explícita → <dir_yaml>/bzImage → /mnt/nkr/kernel/bzImage → junto al binario nkr
Initramfs	Explícita → por nombre de servicio → por nombre de disco → heurística → auto-generación

10.3 Funcionalidades

- **Auto-build:** Si un servicio tiene una sección `build:` y el disco no existe, se construye automáticamente
- **Health checks:** Monitorización TCP con retardo, intervalo y reintentos configurables
- **Modo daemon:** `nkr compose up -d` ejecuta en segundo plano con rotación de logs (máx. 10 MB, 3 rotados)
- **Snapshots CoW:** Creación automática de snapshots cuando un disco base ya está en uso por otra VM
- **IDs deterministas:** Los servicios se ordenan alfabéticamente; los IDs se asignan de forma determinista mediante un registro persistente (`/mnt/nkr/registry.json`)

10.4 Directorio de Datos NKR

```

/mnt/nkr/
├─ images/
├─ initramfs/
│   ├─ base/
│   └─ pg.cpio.gz
│   └─ odoo.cpio.gz
├─ kernel/
└─ snapshots/
# Default (variable NKR_DATA_DIR)
# Imágenes de disco ext4 base
# Ficheros .cpio.gz por servicio
# busybox + módulos del kernel (compartido)
# bzImage compartido
# Snapshots CoW por stack

```

└─ registry.json # Mapa persistente nombre → ID

11 Despliegue Multi-Tenant

NKR incluye un conjunto de herramientas de despliegue completo para Odoo 17 multi-tenant.

11.1 Registro de Clientes

Los clientes se definen en `deploy/clients.yml`:

```
global:
  pg_ram: 2048
  odoo_ram: 256
  odoo_chrs: 1
  base_disk: /opt/nkr/discos/odoo-base.ext4
  db_statement_timeout: 60000 # ms – duración máxima de query por tenant (nuevo v1.1)
  db_conn_limit: 10          # conexiones simultáneas máx por base de datos (nuevo v

clients:
  - name: acme
    domain: acme.ejemplo.com
    db_name: acme_prod
  - name: globex
    domain: globex.ejemplo.com
    db_name: globex_prod
    ram: 512 # override
    chrs: 2 # override
    db_conn_limit: 20 # override – cliente con mayor carga
```

11.2 Pipeline de Aprovisionamiento

`mt-provision.sh <nombre-cliente>`

```
|
├─ Crear disco CoW:    cp --reflink=auto base.ext4 → <cliente>.ext4
├─ Generar config Odoo: odoo.conf con db_name, admin_passwd, workers=0
├─ Generar config nginx: <dominio> → 10.0.0.<ip_vm>:8069
├─ Activar sitio nginx: ln -s sites-available → sites-enabled
```

```

└─ Recargar nginx:      nginx -s reload
└─ Límites PostgreSQL (nuevo v1.1):
    └─ ALTER DATABASE "<db>" SET statement_timeout = '<N>ms';
    └─ ALTER DATABASE "<db>" CONNECTION LIMIT <N>;

```

Los límites de base de datos se aplican directamente sobre PostgreSQL vía psql en 10.0.0.2:5432, con espera activa de hasta 30 intentos para garantizar que PG esté disponible. Si no responde, el script emite un aviso (soft fail) sin interrumpir el aprovisionamiento.

11.3 Generación Automática de Compose — Nuevo en v1.1

El script `deploy/mt-compose-gen.sh` genera `nkr-compose.yml` de forma determinista a partir de `clients.yml`, eliminando la gestión manual de IDs y puertos:

```

sudo ./deploy/mt-compose-gen.sh          # escribe nkr-compose.yml
sudo ./deploy/mt-compose-gen.sh --dry-run # imprime sin escribir

```

Reglas de asignación:

Servicio	ID	Puertos
PostgreSQL	1	5432:5432
Cliente #1 (primer en clients.yml)	2	8069:8069, 8072:8072
Cliente #2	3	8070:8069, 8073:8072
Cliente #N	N+1	(8069+N-1):8069, (8072+N-1):8072

El script es idempotente: sobrescribe `nkr-compose.yml` en cada ejecución. El orden de los clientes en el YAML de salida es el mismo que en `clients.yml`, garantizando IDs estables ante re-ejecuciones.

11.4 Actualización de Módulos en Caliente (Hot Module Update)

El script `deploy/update.sh` proporciona actualizaciones de módulos con tiempo de inactividad casi nulo:

Modo	Comando	Tiempo inactivo
Producción	<code>update.sh</code>	~5 segundos
Test	<code>update.sh --test</code>	0 (ejecuta en puerto 8070)

Modo	Comando	Tiempo inactivo
Rollback	<code>update.sh --rollback</code>	~5 segundos
Actualizar BD	<code>update.sh --update-db</code>	~30 segundos

Flujo de actualización:

1. Copia de seguridad automática de módulos actuales (conserva las últimas 5)
2. Detener VM Odoo (PostgreSQL sigue ejecutándose)
3. Montar disco, rsync de módulos con `__manifest__.py`
4. Reiniciar VM Odoo → ~5s de tiempo inactivo total

11.5 Arquitectura Objetivo

Servidor (16–32 GB RAM)

```

|
├─ 1× VM PostgreSQL (1–2 GB RAM, 2 chrs)
|   └─ Todas las bases de datos de los 50 clientes
|
├─ 50× VMs Odoo (~256 MB c/u, 1 chr)
|   ├── acme      (id=2, 10.0.0.3:8069)
|   ├── globex    (id=3, 10.0.0.4:8069)
|   └─ ...
|
├─ nginx (en el host, no en VM)
|   └─ Proxy inverso + SSL por dominio
|
└─ Puertos expuestos: 80, 443, 5566 (SSH)
    Todo lo demás: interno en nkr0

```

Escalabilidad de densidad (servidor 32 GB):

Escenario	RAM/Instancia	Instancias en 32 GB
v1.1 base	~640 MB	50
v1.2 + PMEM	~440 MB	72
v1.2 + PMEM + KSM	~330 MB	96

Escenario	RAM/Instancia	Instancias en 32 GB
v1.2 todas las features + KSM	~310 MB	103

Presupuesto de recursos para 50 inquilinos (v1.2):

Componente	RAM	Disco
PostgreSQL	2 GB	2 GB (compartido)
50× Odoo (con PMEM)	50 × ~440 MB ≈ 22 GB	~4 GB base + deltas CoW
Sobrecarga NKR	~50 × 5 MB ≈ 250 MB	~3 MB binario
Total	~24 GB	~10 GB (vs ~75 GB Docker)

Con KSM activado, la RAM efectiva por instancia Odoo baja a ~330 MB (ahorro adicional del 25-30% por compartición de páginas idénticas de Python/librerías entre VMs).

12 Observabilidad y Métricas

NKR incorpora un sistema de telemetría de bajo nivel implementado en el propio hipervisor que mide y expone los recursos utilizados en tiempo real por cada micro-VM, evitando la necesidad de desplegar agentes adicionales dentro de los guests.

El motor de métricas extrae información mediante sondas ligeras desde procfs y del subsistema de red del host:

- **CPU%:** Calculado mediante una ventana de muestreo síncrona de 200 ms analizando `/proc/{pid}/stat`. Diseñado para amortiguar el impacto computacional, el intervalo se comparte de forma global si se verifican múltiples VMs simultáneas.
- **RAM (VmRSS):** NKR audita la memoria física real (RSS) utilizada en el servidor leyendo de `/proc/{pid}/status`. Permitiendo una visualización certera y sin overhead de la cantidad de megabytes liberados u ocupados contra los megabytes pre-asignados a la VM.
- **Disco (E/S):** Bytes acumulados de lecturas y escrituras en el bloque raíz y bases de datos (`/proc/{pid}/io`).

- **Red (TAP):** Transferencia y recepción volumétrica de la interfaz de red emulada (TAP) para fiscalizar el ancho de banda cruzado entre el huésped y el exterior usando `/proc/net/dev`.
- **Estado KSM (Kernel Same-page Merging):** Supervisión instantánea del deduplicador de páginas en memoria de Linux. La CLI de NKR computa los “MB ahorrados” cuantificando en tiempo real con qué proporción coinciden páginas idénticas entre micro-VMs Odoos — crucial para sostener la extrema hiperdensidad.

Toda la información anterior puede ser consultada en consola, agrupando de manera tabular las estadísticas de inquilinos de la forma:

```
sudo nkr stats
```

12.1 Exportador Prometheus Nativo — Nuevo en v1.1

NKR v1.1 incluye un servidor de métricas Prometheus integrado en el binario, sin dependencias externas. Se activa con el subcomando `serve`:

```
sudo nkr serve --port 9090
# Expone: http://host:9090/metrics
```

El endpoint implementa el formato de exposición Prometheus 0.0.4 (text/plain) usando únicamente `std::net::TcpListener` de la biblioteca estándar de Rust (~30 líneas). No requiere ningún crate adicional.

Métricas expuestas:

Métrica	Tipo	Descripción
<code>nkr_cpu_pct{vm="..."}</code>	Gauge	Porcentaje de CPU consumido (ventana 50 ms)
<code>nkr_rss_mb{vm="..."}</code>	Gauge	RAM física real (RSS) en MB
<code>nkr_io_read_bytes{vm="..."}</code>	Counter	Bytes leídos del disco (acumulado)
<code>nkr_io_write_bytes{vm="..."}</code>	Counter	Bytes escritos al disco (acumulado)
<code>nkr_net_rx_bytes{vm="..."}</code>	Counter	Bytes recibidos por el TAP (acumulado)
<code>nkr_net_tx_bytes{vm="..."}</code>	Counter	Bytes enviados por el TAP (acumulado)
<code>nkr_ksm_savings_mb</code>	Gauge	MB ahorrados globalmente por KSM

Este endpoint es directamente compatible con Grafana Prometheus datasource. La ventana de medición se reduce a 50 ms para scrapes frecuentes sin incrementar la

carga del host de forma significativa.

Esta telemetría directa provee visibilidad total e instantánea sobre los costos infraestructurales consumidos por cliente a un bajísimo impacto en los recursos del servidor.

13 Comparación con Soluciones Existentes

13.1 NKR vs Docker

Dimensión	Docker	NKR
Aislamiento	Kernel compartido (namespaces + cgroups)	VM completa (KVM, kernel separado)
Vulnerabilidad de kernel	Afecta a todos los contenedores	Afecta solo a la VM con ese kernel
Garantía de CPU	Shares de cgroups (límite suave)	Core pinning (límite duro)
RAM	Overcommit por defecto	Exclusiva, sin overcommit
Tamaño del binario	dockerd ~100 MB + containerd + runc	~2-4 MB binario único
Tiempo de arranque	~1-3 segundos (inicio de proceso)	~1-2 segundos (arranque de VM)
Formato de disco	Overlay filesystem por capas	ext4 en bruto (snapshots CoW)
Red	veth + bridge	TAP + bridge + iptables
Compose	docker-compose (YAML)	nkr compose (YAML, sintaxis compatible)

13.2 NKR vs Firecracker

Dimensión	Firecracker	NKR
Lenguaje	Rust	Rust
Interfaz KVM	Directa (kvm-ioctls)	Directa (kvm-ioctls)

Dimensión	Firecracker	NKR
VirtIO	MMIO	MMIO
Enfoque	Serverless (AWS Lambda)	SaaS multi-tenant (Odoo)
Gestión de discos	Externa	Integrada (nkr pull/build, OCI→ext)
Orquestación	Ninguna (externa: containerd)	Integrada (nkr compose)
Herramientas MT	Ninguna	Completas (clients.yml, aprovisiona)
Inyección de volúmenes	Externa	Integrada (montaje pre-boot)
Modelo de CPU	vCPU estándar	«Chrs» (granularidad del 20% con p)
Líneas de código	~70.000+	~5.700 (alcance enfocado)

13.3 NKR vs QEMU/KVM

Dimensión	QEMU/KVM	NKR
Tamaño del binario	~20-50 MB	2-4 MB
Modelo de dispositivo	Emulación x86 completa (PCI, USB, ACPI...)	Solo VirtIO-MMIO mínimo
Configuración	CLI compleja / XML libvirt	Flags CLI simples / YAML
Tiempo de arranque	~3-10 segundos	~1-2 segundos
Dependencias	libvirt, qemu, virt-manager	Ninguna (solo /dev/kvm)
Superficie de ataque	Grande (emulación completa)	Mínima (3 tipos de dispositivo)

14 Modelo de Seguridad

14.1 Fronteras de Aislamiento

Capa	Mecanismo
CPU	Virtualización hardware KVM (VT-x/AMD-V). El guest ejecuta en ring 0 de un espacio de direcciones separado.
Memoria	GuestMemoryMmap crea regiones de memoria dedicadas. Sin memoria compartida entre VMs.
Disco	Cada VM tiene su propio fichero ext4. Sin overlay filesystem compartido.
Red	Dispositivo TAP separado por VM. Reglas iptables por VM. Reglas ebtables L2 (v1.1): solo la MAC+IP asignada por el hypervisor puede emitir tráfico desde ese TAP.
Proceso	Cada VM se ejecuta como un proceso host separado con su propio PID. SIGTERM/SIGKILL para el ciclo de vida.
Syscalls	Jailer Seccomp BPF (v1.2) restringe el proceso vCPU a ≤ 31 syscalls permitidas tras la inicialización.

14.2 Superficie de Ataque

La superficie de ataque de NKR es significativamente menor que la de Docker y QEMU:

- **Sin emulación de dispositivos en espacio de usuario** (vs QEMU): solo 5 manejadores MMIO (net, block, 9P, PMEM + serie)
- **Sin kernel compartido** (vs Docker): un exploit del kernel en el guest no afecta al host
- **Sin rutas de escape de contenedor**: sin namespaces, cgroups ni compartición de procfs
- **Interacción mínima con el host**: solo E/S de ficheros (disco/mmap), lectura/escritura TAP (red) y salida serie
- **Aislamiento L2** (v1.1): reglas ebtables previenen IP/MAC spoofing entre VMs

de inquilinos en el bridge

14.3 Jailer Seccomp BPF — Nuevo en v1.2

Implementación: `seccomp.rs` — ~170 líneas

Antes de entrar en el bucle de ejecución del vCPU, NKR instala un programa `SECCOMP_MODE_FILTER` construido en tiempo de ejecución a partir de una allowlist estática de 31 syscalls. El filtro usa `libc::prctl` directamente, sin dependencias adicionales.

- **Preámbulo:** `prctl(PR_SET_NO_NEW_PRIVS, 1)` (requerido por el kernel antes de instalar el filtro)
- **Política:** `SECCOMP_RET_KILL_PROCESS` para cualquier syscall fuera de la allowlist
- **Allowlist incluye:** `read`, `write`, `ioctl` (KVM ioctls), `mmap`, `madvise`, `clone` (`thread::spawn`), `futex`, `io_uring_*`, `epoll_*`, `eventfd2`, `openat`, `pread64/pwrite64`, `clock_gettime`, `exit_group` y esenciales de `stdlib`
- **Timing:** Se instala después de `VirtioNetDevice::new()` (que hace `spawn` del hilo RX) para que el hilo de fondo exista antes de aplicar el filtro al proceso
- **Degradación:** Si `prctl` falla (`kernel < 3.17` o permisos denegados), NKR emite un aviso y continúa sin el filtro

14.4 Seguridad Operacional

- Solo los puertos 80, 443 y SSH (configurable) están expuestos externamente
- Todo el tráfico inter-VM está confinado al bridge `nkr0` (10.0.0.0/24)
- Requiere acceso root para KVM/TAP/iptables (intencionado — sin modo sin root)
- El filtro Seccomp (v1.2) restringe el proceso vCPU a la huella mínima de syscalls tras la inicialización de la VM

15 Limitaciones y Trabajo Futuro

15.1 Limitaciones Actuales

Limitación	Impacto	Resolución Planificada
vCPU único por VM	No se puede usar SMP en los guests	Soporte multi-vCPU (prioridad media)

Limitación	Impacto	Resolución Planificada
Solo VirtIO-MMIO	Sin paso a través de PCI	Suficiente para las cargas de trabajo objetivo
9P sin caché de atributos	Latencia en stat() frecuentes	Implementar cache=loose en versión futura
PMEM requiere soporte en kernel guest	CONFIG_VIRTIO_PMEM=y + CONFIG_FS_DAX=y necesarios	Documentado; degradación silenciosa a VirtIO-Block
Sin migración en vivo	Hay que detener la VM para moverla entre hosts	Trabajo futuro
Sin snapshots en caliente	Hay que detener la VM para hacer snapshot del disco	Trabajo futuro
Sin pruebas automatizadas	Solo pruebas manuales	Suite de pruebas unitarias e integración
Solo host Linux	Requiere Linux con KVM	Por diseño
ebtables opcional	Aislamiento L2 solo si ebtables instalado	Migración a nftables bridge en versión futura

15.2 Hoja de Ruta

Implementado en v1.1:

- Generar `nkr-compose.yml` automáticamente → `mt-compose-gen.sh` □
- VirtIO-FS para directorios compartidos → VirtIO-9P (- -share) □
- Panel de monitorización de recursos → Exportador Prometheus (`nkr serve`) □
- Aislamiento de red entre inquilinos → ebtables L2 isolation □
- Protección de base de datos por tenant → `statement_timeout` + `conn_limit` □
- CPU bursting controlado → `cgroupv2 cpu.max` + `cpu.max.burst` □

Implementado en v1.2 y v1.3:

- Arranque más rápido → cargador ELF `vmlinux` (-20 ms) □
- Reducción de coste de syscalls → io_uring E/S asíncrona (~70% reducción) □
- Reducción de RAM por instancia → VirtIO-PMEM + DAX (-150-200 MB/VM) □

- ~~Alta velocidad IO Host-Guest~~ → VirtIO-FS DAX vhost-user (Reemplazo 9P) □
- ~~Deduplicador RAM idle~~ → VirtIO-Balloon □
- ~~Reducción de superficie de ataque~~ → Jailer Seccomp BPF □

Alta prioridad:

- Pruebas end-to-end con N clientes reales
- Let's Encrypt automatizado vía certbot
- Migración a nftables bridge (reemplazar ebtables)

Prioridad media:

- Soporte de múltiples vCPUs
- Caché de atributos 9P (cache=loose) para reducir latencia en stat() frecuentes
- Copia de seguridad automatizada de PostgreSQL por inquilino

Prioridad baja:

- Migración en vivo entre servidores
 - Snapshots en caliente sin detener la VM
 - Interfaz de gestión web
-

16 Conclusión

NKR demuestra que es posible conseguir **densidad y simplicidad operacional a nivel de contenedor** con **aislamiento y garantías de recursos a nivel de VM**, en menos de 5.500 líneas de Rust, compilando a un binario de 2-4 MB sin dependencias en tiempo de ejecución.

La versión 1.2 eleva el techo de densidad de 50 a 100+ instancias Odoo en un servidor de 32 GB mediante cuatro optimizaciones que se combinan: VirtIO-PMEM elimina la caché de páginas duplicada (~150-200 MB ahorrados por VM), io_uring reduce el coste de syscalls ~70% bajo alta concurrencia, la carga ELF vmlinux ahorra ~20 ms por arranque, y el jailer Seccomp bloquea el proceso vCPU a la huella mínima de syscalls tras la inicialización.

Para los operadores que gestionan docenas de inquilinos SaaS en un único servidor, NKR ofrece un equilibrio fundamentalmente distinto al de Docker o las VMs tradicionales:

- **Cada inquilino obtiene aislamiento hardware**, no solo separación de namespaces

- **Cada inquilino obtiene recursos garantizados**, no pools compartidos de CPU y memoria
- **El operador mantiene los flujos de trabajo de Docker**, con patrones familiares de build, run y compose
- **La infraestructura se consolida**: 1 PostgreSQL, N instancias Odoo, 1 nginx; en vez de N stacks completos

NKR es software con propósito. En lugar de intentar ser un hipervisor de propósito general como QEMU o una plataforma de contenedores de propósito general como Kubernetes, NKR se enfoca en un patrón de carga de trabajo específico y de alto valor: **SaaS multi-tenant denso sobre bare metal**. Este enfoque le permite ser lo suficientemente simple como para comprenderse completamente, lo suficientemente pequeño como para auditarse línea a línea, y lo suficientemente rápido como para arrancar en segundos.

17 Apéndice A: Stack Tecnológico

Componente	Tecnología	Versión	Desde
Lenguaje	Rust	Edition 2021	v1.0
Interfaz KVM	kvm-ioctls	0.19	v1.0
Bindings KVM	kvm-bindings	0.10	v1.0
Memoria del guest	vm-memory (GuestMemoryMmap)	0.14	v1.0
Cargador de kernel	linux-loader (bzImage + ELF)	0.11	v1.0 / v1.2
Colas VirtIO	virtio-queue	0.12	v1.0
CLI	clap (derive)	4.x	v1.0
Serialización	serde + serde_yaml + serde_json	1.x / 0.9 / 1.x	v1.0
Utilidades del sistema	vmm-sys-util	0.12	v1.0
E/S asíncrona	io-uring	0.6	v1.2
Kernel del guest	Linux bzImage / vmlinux	6.6.117-0-virt	v1.0

18 Apéndice B: Métricas del Código Fuente

Módulo	Fichero	Líneas	Responsabilidad
Motor VMM	vmm.rs	~1.400	Init KVM, PIT2, cargador ELF/bzImage, MMIO, cgroups, ebtables, slot PMEM, seccomp
Compose	compose.rs	840	YAML, orquestación, health checks, modo daemon

Módulo	Fichero	Líneas	Responsabilidad
Compartir FS	virtio_fs.rs	~200	Soporte de VirtIO-FS (DAX, vhost-user) en sustitución a 9P. (v1.3)
Globos	balloon.rs	~150	Liberación activa MADV_DONTNEED de páginas no usadas con VirtIO-Balloon (v1.3)
9P Server	p9.rs	~490	Servidor de legado / fallback VirtIO-9P2000.L (v1.1)
Initramfs	initramfs.rs	~410	Boot envs, módulos FS/PMEM (actualizado v1.1/v1.2)
Métricas	metrics.rs	~420	Telemetría /proc, KSM, exportador Prometheus (v1.1)
Red	net.rs	~310	VirtIO-net, backend TAP, hilos RX/TX, io_uring TX (v1.2)
Bloque	block.rs	~310	VirtIO-block, io_uring E/S asíncrona + fallback síncrono (v1.2)
Estado	state.rs	252	Registro de VMs, tracking ciclo de vida, nkr ps

Módulo	Fichero	Líneas	Responsabilidad
PMEM	<code>pmem.rs</code>	~200	VirtIO-PMEM + DAX, mmap(MAP_SHARED), manejador FLUSH (v1.2)
Seccomp	<code>seccomp.rs</code>	~170	Construcción filtro BPF + instalación vía prctl (v1.2)
Pull	<code>pull.rs</code>	201	Pipeline Docker Hub → ext4
Build	<code>build.rs</code>	192	Pipeline Nkrfile → ext4
Registry	<code>registry.rs</code>	166	Asignación persistente nombre → ID
CLI	<code>cli.rs</code>	~200	CLI: --share, --pmem, subcomando serve (actualizado)
Main	<code>main.rs</code>	~160	Punto de entrada, dispatch de comandos (actualizado)
Total		~5.700	(+~1.700 líneas respecto a v1.0)

19 Apéndice C: Inicio Rápido

```
# Compilar NKR desde el código fuente
cargo build --release
# Binario: target/release/nkr (~2–4 MB)
```

```
# Descargar una imagen PostgreSQL y crear un disco
sudo ./target/release/nkr pull postgres:15 postgres.ext4 --size-mb 2048

# Ejecutar una micro-VM
sudo ./target/release/nkr run \
  --disk postgres.ext4 \
  --ram 512 \
  --chrs 1 \
  --id 1 \
  --port 5432:5432

# Ejecutar con compartición de sistema de archivos en vivo (v1.1)
sudo ./target/release/nkr run \
  --disk odoo.ext4 \
  --ram 256 --chrs 1 --id 2 \
  --share /opt/modules:/mnt/extra-addons \
  --share /opt/config:/etc/odoo

# Ejecutar con VirtIO-PMEM + DAX (~150–200 MB ahorro de RAM) (v1.2)
# Requiere kernel guest con CONFIG_VIRTIO_PMEM=y y CONFIG_FS_DAX=y
sudo ./target/release/nkr run \
  --disk odoo.ext4 \
  --ram 256 --chrs 1 --id 3 \
  --pmem

# Arrancar con kernel ELF vmlinux sin comprimir (~20 ms más rápido) (v1.2)
sudo ./target/release/nkr run \
  --kernel /boot/vmlinux \
  --disk odoo.ext4 \
  --ram 256 --chrs 1 --id 4

# Listar VMs en ejecución
sudo ./target/release/nkr ps

# Ver estadísticas de recursos
sudo ./target/release/nkr stats

# Iniciar exportador de métricas Prometheus (v1.1)
```

```
sudo ./target/release/nkr serve --port 9090
# Consultar: curl http://localhost:9090/metrics

# Activar KSM para deduplicación de páginas entre VMs
sudo ./target/release/nkr ksm on

# Detener una VM
sudo ./target/release/nkr stop 1

# Generar compose multi-tenant automáticamente (v1.1)
sudo ./deploy/mt-compose-gen.sh

# Orquestrar un stack multi-servicio
sudo ./target/release/nkr compose up -f nkr-compose.yml -d
```

NKR es software de código abierto. Las contribuciones y comentarios son bienvenidos.

© 2026 NKR Contributors. Licencia MIT.